

DOCUMENTACIÓN TÉCNICA

Recicley Watermark

[Manual de uso y explicación del código fuente](#)

Archivo fuente documentado: app/GUI_Watermark.py

Framework de interfaz: PyQt6

Motor de marca de agua: PyPDF2 + reportlab + Pillow (sin dependencias externas)

Plataforma: Windows 10/11 (64 bits)

Fecha del documento: 4 de agosto de 2026

Índice

1. Descripción general
2. Instalación y requisitos
3. Guía de uso de la aplicación
4. Arquitectura general del código
5. Módulos importados y constantes globales
6. La clase DragDropWindow — construcción de la interfaz
7. Gestión del estado: lista de PDFs, marca de agua y destino
8. Diálogos de selección de archivos
9. Arrastrar y soltar (Drag & Drop)
10. processWatermark — el orquestador del procesamiento por lotes
11. unique_output_name y addWaterMarkToPDF — el motor que aplica la marca de agua
12. Punto de entrada main()
13. Flujo de datos completo (paso a paso)
14. Empaquetado a .exe (PyInstaller)
15. Solución de problemas

1. Descripción general

Recicley Watermark es una aplicación de escritorio para Windows, construida en Python con la librería gráfica PyQt6, que permite aplicar una imagen de marca de agua (con transparencia) sobre uno o varios archivos PDF de forma idéntica. El usuario carga los PDF (arrastrándolos, seleccionándolos manualmente o apuntando a una carpeta completa), carga una imagen de marca de agua, ajusta la opacidad deseada y, con un solo clic, la aplicación genera copias marcadas de todos los documentos.

Internamente ya no se usa Poppler. La marca de agua se genera como un PDF vectorial de una sola página (con reportlab), del tamaño exacto de cada página a marcar, y se superpone directamente sobre la página original usando PyPDF2 (`page.merge_page(overlay)`). Solo la imagen de la marca de agua se rasteriza (con Pillow, para poder aplicarle opacidad); el contenido original del PDF — texto, vectores, formularios — nunca se convierte a imagen, así que el documento resultante conserva el mismo texto seleccionable y exactamente el mismo número de páginas que el original. Todo el proceso ocurre dentro del mismo proceso de Python, sin subprocesos externos ni archivos temporales en disco.

2. Instalación y requisitos

Uso como ejecutable (recomendado)

No requiere Python ni ninguna instalación. Entra a la carpeta `GUI_Watermark ejecutable/` del proyecto y ejecuta `GUI_Watermark.exe` directamente. La carpeta `_internal` debe permanecer junto al ejecutable: ahí viven las DLL de PyQt6, el ícono de la aplicación y las demás dependencias empaquetadas. A diferencia de versiones anteriores, ya **no se empaqueta ningún binario de Poppler**, porque el motor de marca de agua actual no depende de él.

Importante: nunca copies solo el .exe a otra máquina sin la carpeta `_internal`; la aplicación no podrá iniciar porque le faltarán sus dependencias. Para compartir la app, copia la carpeta `GUI_Watermark ejecutable/` completa, o comparte el .zip incluido en ella.

Ejecución desde el código fuente

Requisito	Uso
Python 3.13	Intérprete del lenguaje.
PyQt6	Interfaz gráfica (ventana, botones, slider, diálogos, drag & drop).
PyPDF2	Lee/escribe el PDF de origen página por página y fusiona (<code>merge_page</code>) el overlay de la marca de agua directamente sobre cada página original.
Pillow (PIL)	Redimensiona la imagen de marca de agua y ajusta su canal alfa según la opacidad elegida.
reportlab	Genera en memoria un PDF vectorial de una sola página, del tamaño exacto de la página a marcar, que sirve como overlay de la marca de agua.

No se requiere Poppler ni ningún otro binario externo: la marca de agua se superpone directamente sobre cada página, vectorialmente y sin rasterizar el PDF original.

3. Guía de uso de la aplicación

- 1 **Cargar PDF(s):** arrastra uno o varios archivos .pdf, o una carpeta completa (se agregan recursivamente todos los PDF que contenga), a la ventana. También puedes usar el botón **Archivos** (selector múltiple) o **Carpeta** (selector de carpeta). El botón **Limpiar** vacía la lista.
- 2 **Cargar la marca de agua:** arrastra una imagen .png/.jpg/.jpeg o usa el botón **Elegir imagen**. Se recomienda un PNG con canal alfa (fondo transparente) para un resultado más limpio.
- 3 **Elegir destino (opcional):** por defecto los archivos resultantes van a la carpeta Descargas del usuario. El botón **Elegir destino** permite cambiarla.
- 4 **Ajustar opacidad:** el control deslizante define qué tan visible es la marca de agua, de 0 (invisible) a 255 (totalmente opaca).
- 5 **Procesar:** al pulsar **Agregar Marca de Agua**, se recorre cada PDF de la lista, mostrando una barra de progreso cancelable. Al terminar se muestra un resumen y se abre automáticamente la carpeta de salida en el explorador de Windows.

4. Arquitectura general del código

El código fuente vive en `app/GUI_Watermark.py`, organizado en tres bloques conceptuales:

- 1 **Configuración global:** imports, la constante `RENDER_DPI`, y la función auxiliar `get_base_path()`, usada para localizar el ícono empaquetado tanto en modo script como en modo ejecutable.
- 2 **La interfaz gráfica y su lógica:** la clase `DragDropWindow(QMainWindow)`, que concentra tanto la construcción visual de la ventana como todos los manejadores de eventos (clics, arrastre, progreso) y el motor de marca de agua.
- 3 **El punto de entrada:** la función `main()`, que crea la aplicación Qt y muestra la ventana.

No existe separación en múltiples archivos ni patrón MVC formal: al ser una utilidad de escritorio pequeña, `DragDropWindow` actúa a la vez como vista (widgets) y controlador (lógica de negocio), delegando el procesamiento de imágenes y PDF a tres librerías externas: Pillow, reportlab y PyPDF2. El proyecto además se reorganizó en carpetas: `app/` (código fuente, .spec e ícono), `docs/` (esta documentación) y `GUI_Watermark ejecutable/` (el build ya listo para distribuir, con su .exe, _internal/ y .zip portable).

5. Módulos importados y constantes globales

```
import sys
import os
import io
from datetime import datetime
from PyQt6.QtWidgets import (
    QApplication, QMainWindow, QLabel, QWidget, QPushButton, QMessageBox,
    QSlider, QFileDialog, QProgressDialog
)
from PyQt6.QtGui import QIcon
from PyQt6.QtCore import Qt

from PyPDF2 import PdfWriter, PdfReader
from PIL import Image
from reportlab.pdfgen import canvas as reportlab_canvas
from reportlab.lib.utils import ImageReader
```

Módulo	Para qué se usa en este proyecto
sys	Acceso a <code>sys.argv</code> (arranque de la app Qt), <code>sys.exit()</code> y detección de si el programa corre "congelado" (empaquetado como <code>.exe</code>) vía <code>sys.frozen/sys._MEIPASS</code> .
os	Rutas de archivos y carpetas: unir rutas, listar/recurrer directorios (<code>os.walk</code>), obtener la carpeta de usuario, crear carpetas, abrir el explorador (<code>os.startfile</code>).
io	Buffers de memoria (<code>io.BytesIO</code>) para generar el overlay de marca de agua y el PDF final sin escribir archivos temporales en disco.
datetime	Genera la marca de tiempo usada en el nombre de la subcarpeta de salida cuando se procesan varios PDFs a la vez.
PyQt6.QtWidgets	Todos los widgets de la interfaz: ventana principal, etiquetas, botones, slider, cuadros de diálogo de archivo/carpeta y la barra de progreso.
PyQt6.QtGui.QIcon	Carga el ícono de la ventana (<code>rec.jpg</code>).
PyQt6.QtCore.Qt	Constantes de Qt: orientación del slider, modalidad de la ventana de progreso.
PyPDF2.PdfWriter / PdfReader	Lectura del PDF original página por página, lectura del overlay generado en memoria, y escritura del PDF final con las páginas ya marcadas.
PIL.Image	Redimensiona la marca de agua y ajusta su canal alfa según la opacidad elegida antes de dibujarla en el overlay.
reportlab.pdfgen.canvas	Dibuja la imagen de marca de agua (ya con su alfa aplicado) sobre un lienzo PDF del tamaño exacto de la página, generando el overlay vectorial de una sola hoja.
reportlab.lib.utils.ImageReader	Adaptador que permite pasarle a <code>canvas.drawImage</code> una imagen de Pillow directamente, sin guardarla a disco.

```

RENDER_DPI = 200
IMAGE_EXTENSIONS = ('.png', '.jpg', '.jpeg')

```

`RENDER_DPI` ya no controla la resolución de la página completa (no se rasteriza), sino únicamente la resolución a la que se rasteriza la **imagen de la marca de agua** antes de incrustarla en el overlay vectorial: a mayor DPI, más nítida se ve la marca de agua, a costa de un overlay más pesado. La constante `PDF_SAVE_QUALITY` de versiones anteriores desapareció por completo: ya no existe una re-codificación JPEG de cada página, así que ese parámetro de calidad dejó de tener sentido. `IMAGE_EXTENSIONS` centraliza las extensiones aceptadas como imagen de marca de agua, para no repetirlas en cada validación.

función `get_base_path()`

```

def get_base_path():
    if getattr(sys, 'frozen', False):
        return getattr(sys, '_MEIPASS', os.path.dirname(sys.executable))
    return os.path.dirname(os.path.abspath(__file__))

```

Devuelve la carpeta desde la que debe leerse el recurso empaquetado (`rec.jpg`, el ícono de la ventana). Como ya no se empaqueta Poppler, esta función solo necesita resolver esa única carpeta de recursos. Cuando el programa corre como script normal, es la carpeta donde está `GUI_Watermark.py`. Cuando corre como `.exe` generado por PyInstaller (`sys.frozen` es `True`), usa `sys._MEIPASS`: la carpeta interna (`_internal`) donde PyInstaller coloca los datos empaquetados.

6. La clase DragDropWindow — construcción de la interfaz

`DragDropWindow(QMainWindow)` hereda de `QMainWindow` y representa la única ventana de la aplicación (sin cambios respecto a versiones anteriores: la interfaz no se tocó en esta reescritura, solo el motor interno de marca de agua). Su tamaño es fijo (500×520 px, mínimo = máximo, no redimensionable) y cada widget se posiciona con coordenadas absolutas (`setGeometry(x, y, ancho, alto)`).

#	Atributos en el código	Rol
1	<code>title</code>	Título decorativo "Protector de archivos".
2	<code>label_pdf</code> , <code>button_select_files</code> , <code>button_select_folder</code> , <code>button_clear_files</code>	Carga y gestión de la lista de PDF.
3	<code>label_watermark</code> , <code>button_select_watermark</code>	Carga de la imagen de marca de agua.
4	<code>label_output</code> , <code>button_select_output</code>	Carpeta de destino de los resultados.
5	<code>opacity_label</code> , <code>opacity_slider</code>	Opacidad aplicada a la marca de agua (0–255).
6	<code>button</code>	Botón principal; dispara <code>processWatermark()</code> .

Cada botón se conecta a su manejador con la señal `clicked.connect(...)` de Qt, y el slider con `valueChanged.connect(self.updateOpacity)`, siguiendo el patrón estándar de señales/slots de PyQt.

método `updateOpacity(self)`

```
def updateOpacity(self):
    self.opacity = self.opacity_slider.value()
    self.opacity_label.setText(f"Opacidad: {self.opacity}")
```

Se ejecuta cada vez que el usuario mueve el slider. Simplemente refleja el valor numérico actual en la etiqueta de texto, para dar retroalimentación inmediata.

7. Gestión del estado: lista de PDFs, marca de agua y destino

El estado de la aplicación vive en tres atributos de instancia, inicializados en `__init__`:

- `self.pdf_paths`: lista de rutas absolutas a los PDF cargados (puede tener 0, 1 o N elementos).
- `self.watermark_path`: ruta de la imagen de marca de agua (una sola, aplicada a todos los PDF).
- `self.output_dir`: carpeta de destino elegida por el usuario, o cadena vacía si se usará la carpeta por defecto (Descargas).

método `add_pdf(self, path)`

```
def add_pdf(self, path):
    if path not in self.pdf_paths:
        self.pdf_paths.append(path)
```

Agrega una ruta a `pdf_paths` evitando duplicados (por ejemplo, si el usuario arrastra dos veces el mismo archivo, o si aparece tanto al arrastrar un archivo suelto como dentro de una carpeta arrastrada después).

método `add_pdfs_from_folder(self, folder)`

```
def add_pdfs_from_folder(self, folder):
    found = []
    for root, _dirs, files in os.walk(folder):
        for f in files:
            if f.lower().endswith('.pdf'):
                found.append(os.path.join(root, f))
    for f in sorted(found):
        self.add_pdf(f)
```

`os.walk(folder)` recorre la carpeta indicada y todas sus subcarpetas de forma recursiva. Cada archivo cuyo nombre termine en `.pdf` (sin distinguir mayúsculas/minúsculas) se agrega a una lista temporal, que luego se ordena alfabéticamente y se añade una a una vía `add_pdf`. Tanto el botón **Carpeta** como arrastrar una carpeta a la ventana llaman a este mismo método.

update_pdf_label(self) / clearFiles(self)

`update_pdf_label` sincroniza el texto de `label_pdf` (y su tooltip, con la lista completa de nombres) con el contenido real de `self.pdf_paths`, tras cualquier operación que la modifique. `clearFiles` vacía la lista y refresca la etiqueta; está conectado al botón **Limpiar**.

8. Diálogos de selección de archivos

Estos cuatro métodos son manejadores de clic que abren los cuadros de diálogo nativos de Windows (QFileDialog) como alternativa a arrastrar y soltar.

Método	Diálogo de Qt usado	Efecto
<code>selectFileDialog</code>	<code>getOpenFileNames</code> (filtro <code>*.pdf</code>)	Permite elegir varios PDF a la vez; cada uno se agrega con <code>add_pdf</code> .
<code>selectFolderDialog</code>	<code>getExistingDirectory</code>	Elige una carpeta y delega en <code>add_pdfs_from_folder</code> para el escaneo recursivo.
<code>selectWatermarkDialog</code>	<code>getOpenFileName</code> (filtro imágenes)	Reemplaza <code>self.watermark_path</code> por la imagen elegida.
<code>selectOutputDialog</code>	<code>getExistingDirectory</code>	Fija <code>self.output_dir</code> con la carpeta elegida y actualiza <code>label_output</code> .

En los cuatro casos, tras modificar el estado se actualiza inmediatamente el texto del QLabel correspondiente, para que el usuario tenga confirmación visual de qué quedó cargado.

9. Arrastrar y soltar (Drag & Drop)

método dragEnterEvent(self, event)

```
def dragEnterEvent(self, event):
    if event.mimeData().hasUrls():
        for url in event.mimeData().urls():
            file_path = url.toLocalFile()
            if (os.path.isdir(file_path)
                or file_path.lower().endswith('.pdf')
                or file_path.lower().endswith(IMAGE_EXTENSIONS)):
                event.acceptProposedAction()
                break
```

Qt llama a este método continuamente mientras el usuario arrastra algo sobre la ventana, antes de soltarlo. Si al menos uno de los elementos arrastrados es una carpeta, un PDF o una imagen soportada, se acepta la acción.

método `dropEvent(self, event)`

```
def dropEvent(self, event):
    for url in event.mimeData().urls():
        file_path = url.toLocalFile()
        if os.path.isdir(file_path):
            self.add_pdfs_from_folder(file_path)
        elif file_path.lower().endswith('.pdf'):
            self.add_pdf(file_path)
        elif file_path.lower().endswith(IMAGE_EXTENSIONS):
            self.watermark_path = file_path
            self.label_watermark.setText(...)
    self.update_pdf_label()
```

Recorre todas las URLs soltadas y clasifica cada una: si es una carpeta, escanea sus PDF; si es un PDF, lo agrega a la lista; si es una imagen, se convierte en la nueva marca de agua (la última soltada es la que queda). Al final se refresca la etiqueta de PDFs una sola vez.

10. processWatermark — el orquestador del procesamiento por lotes

Este es el método más largo de la clase y actúa como director de todo el proceso cuando el usuario pulsa **Agregar Marca de Agua**.

Puede dividirse en cinco etapas:

10.1 Validación

```
if not self.pdf_paths:
    QMessageBox.warning(...); return
if not self.watermark_path:
    QMessageBox.warning(...); return
```

Si falta el/los PDF o la imagen de marca de agua, se muestra un aviso y se corta la ejecución antes de tocar el disco.

10.2 Cálculo de la carpeta de salida

```
base_output = self.output_dir or os.path.join(os.path.expanduser("~"), "Downloads")
if len(self.pdf_paths) == 1:
    output_dir = base_output
else:
    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    output_dir = os.path.join(base_output, f"Recicley_Watermark_{timestamp}")
    os.makedirs(output_dir, exist_ok=True)
```

Si solo hay un PDF, los resultados van directo a la carpeta de destino. Si hay varios, se crea una subcarpeta con marca de tiempo para no mezclar resultados de distintas ejecuciones.

10.3 Bucle principal con barra de progreso


```

for i, pdf_path in enumerate(self.pdf_paths):
    if progress.wasCanceled():
        break
    progress.setLabelText(f"Procesando: {filename} ({i + 1}/{total})")
    progress.setValue(i)
    QApplication.processEvents()

    out_name = self.unique_output_name(
        output_dir, used_names,
        f"{os.path.splitext(filename)[0]}_marca_de_agua.pdf")
    output_path = os.path.join(output_dir, out_name)

    try:
        self.addWaterMarkToPDF(
            pdf_path, self.watermark_path, opacity, output_path)
        processed += 1
    except Exception as e:
        errors.append(f"{filename}: {e}")

```

Nota respecto a versiones anteriores: `addWaterMarkToPDF` ya no recibe un `poppler_path` (ese parámetro desapareció junto con la dependencia de Poppler), y `unique_output_name` ahora recibe también `output_dir` (ver sección 11). Cada PDF se procesa dentro de un `try/except` individual: si uno falla, el error se guarda y el bucle continúa con el siguiente archivo.

10.4 y 10.5 Nombres sin colisión y resultado final

Antes de procesar cada archivo se calcula su nombre de salida con `unique_output_name` (sección 11). Al terminar, si se procesó al menos un archivo se abre la carpeta de salida con `os.startfile(output_dir)`, y se informa el resultado: si hubo errores se listan junto con los éxitos; si todo salió bien, se confirma la cantidad de archivos procesados y la ruta de salida.

11. `unique_output_name` y `addWaterMarkToPDF` — el motor que aplica la marca de agua

método estático `unique_output_name(output_dir, used_names, filename)`

```

@staticmethod
def unique_output_name(output_dir, used_names, filename):
    base, ext = os.path.splitext(filename)
    candidate = filename
    i = 1
    while (candidate.casefold() in used_names
           or os.path.exists(os.path.join(output_dir, candidate))):
        candidate = f"{base} ({i}){ext}"
        i += 1
    used_names.add(candidate.casefold())
    return candidate

```

Respecto a versiones anteriores, este método ahora recibe también `output_dir` y compara en minúsculas (`casefold()`): como Windows es insensible a mayúsculas/minúsculas en nombres de archivo, dos PDF de entrada como "Factura.pdf" y "factura.pdf" (por ejemplo, en subcarpetas distintas de una misma carpeta arrastrada) producirían el mismo nombre de salida en disco aunque el set en memoria los viera como strings distintos; sin este ajuste el segundo sobrescribiría silenciosamente al primero. También se comprueba `os.path.exists` para no chocar con un archivo que ya exista físicamente en la carpeta de destino de una corrida anterior.

método estático `_resample_filter()`

```
@staticmethod
def _resample_filter():
    try:
        return Image.Resampling.LANCZOS
    except AttributeError:
        return Image.ANTIALIAS
```

Filtro de remuestreo usado al redimensionar la imagen de marca de agua (LANCZOS, con fallback a ANTIALIAS para compatibilidad con versiones antiguas de Pillow). Antes se usaba para redimensionar la marca de agua al tamaño de la página rasterizada completa; ahora cumple exactamente el mismo rol, pero sobre una imagen mucho más pequeña en memoria (solo la marca de agua).

método `_build_watermark_overlay_bytes(watermark_path, opacity, width_pt, height_pt)`

Genera, en memoria, un PDF de una sola página del tamaño exacto de la página a marcar (en puntos PDF) que contiene únicamente la imagen de la marca de agua con su transparencia y opacidad ya aplicadas. No usa Poppler ni ningún proceso externo: todo ocurre en este mismo proceso Python, sin archivos temporales.

```
px_width = max(1, round(width_pt / 72 * RENDER_DPI))
px_height = max(1, round(height_pt / 72 * RENDER_DPI))

waterMark = Image.open(watermark_path).convert("RGBA")
watermark_resized = waterMark.resize((px_width, px_height), cls._resample_filter())
original_alpha = watermark_resized.getchannel("A")
scaled_alpha = original_alpha.point(lambda a, opacity=opacity: a * opacity // 255)
watermark_resized.putalpha(scaled_alpha)

buffer = io.BytesIO()
c = reportlab_canvas.Canvas(buffer, pagesize=(width_pt, height_pt))
c.drawImage(ImageReader(watermark_resized), 0, 0,
            width=width_pt, height=height_pt, mask='auto')
c.save()
return buffer.getvalue()
```

La marca de agua se rasteriza a RENDER_DPI para verse nítida, aunque el tamaño en el que se dibuja sobre la página (width_pt/height_pt) esté definido en puntos PDF (1/72 de pulgada). El canal alfa original se escala proporcionalmente por la opacidad elegida en el slider (`a * opacity // 255`), así que zonas ya transparentes de la imagen siguen siéndolo.

método `addWaterMarkToPDF(pdf_path, watermark_path, opacity, output_path)`

Este es el corazón técnico de la aplicación: superpone la marca de agua sobre cada página del PDF original, **sin rasterizarlas**. El contenido original (texto, vectores, etc.) se conserva intacto; solo se le agrega encima la imagen de la marca de agua.

```
pdf_reader = PdfReader(pdf_path)
pdf_writer = PdfWriter()
overlay_cache = {}

for page in pdf_reader.pages:
    box = page.mediabox
    size = (float(box.width), float(box.height))
    if size not in overlay_cache:
        overlay_cache[size] = cls._build_watermark_overlay_bytes(
            watermark_path, opacity, *size)
    overlay_page = PdfReader(io.BytesIO(overlay_cache[size])).pages[0]
    page.merge_page(overlay_page)
    pdf_writer.add_page(page)

with open(output_path, "wb") as output_pdf:
    pdf_writer.write(output_pdf)
```

El overlay se cachea por tamaño de página: casi todos los PDF tienen el mismo tamaño en todas sus hojas, así que normalmente se genera una sola vez por documento. Se guardan los **bytes** del overlay (no el objeto de página), y se reconstruye un PdfReader nuevo a partir de ellos en cada página que lo usa, para que cada fusión reciba un objeto de página independiente, sin estado compartido entre páginas.

Por qué se abandonó la rasterización completa de la página (el enfoque de versiones anteriores, que convertía cada página a imagen vía Poppler antes de componer la marca de agua): al fusionar el overlay vectorial directamente sobre la página original, el PDF resultante conserva el texto seleccionable, las fuentes y los vectores intactos, y el número de páginas de salida siempre coincide exactamente con el de entrada. Al no depender de un subproceso externo (Poppler) por cada página, también se elimina la causa más probable de páginas repetidas o faltantes en el resultado, y el proceso es considerablemente más rápido y liviano.

12. Punto de entrada main()

```
def main():
    app = QApplication(sys.argv)
    window = DragDropWindow()
    window.show()
    sys.exit(app.exec())

if __name__ == "__main__":
    main()
```

QApplication es el objeto obligatorio de Qt que administra el bucle de eventos, la configuración y los recursos gráficos de toda la aplicación; solo puede existir una instancia. Se crea la ventana DragDropWindow, se muestra con .show() y app.exec() arranca el bucle de eventos hasta que la ventana se cierra; su código de salida se propaga con sys.exit(). Sin cambios respecto a versiones anteriores.

13. Flujo de datos completo (paso a paso)

- 1 El usuario carga PDF(s) → dropEvent / selectFileDialog / selectFolderDialog → add_pdf / add_pdfs_from_folder → self.pdf_paths → update_pdf_label.
- 2 El usuario carga la imagen de marca de agua → dropEvent / selectWatermarkDialog → self.watermark_path.
- 3 (Opcional) El usuario elige destino → selectOutputDialog → self.output_dir.

- 4 El usuario mueve el slider → `updateOpacity` actualiza la etiqueta (el valor real se lee directo del slider al momento de procesar).
- 5 El usuario pulsa "Agregar Marca de Agua" → `processWatermark`: valida `pdf_paths` y `watermark_path`; calcula `output_dir` a partir de `output_dir` o Descargas; por cada PDF, calcula un nombre único (`unique_output_name`) y llama a `addWaterMarkToPDF`.
- 6 `addWaterMarkToPDF` genera (o reutiliza) el overlay vectorial de la marca de agua con reportlab + Pillow, y usa PyPDF2 (`merge_page`) para fusionarlo directamente sobre cada página original — sin rasterizarla — y ensamblar el PDF de salida.
- 7 Al terminar, se abre la carpeta de salida y se informa el resultado.

14. Empaquetado a .exe (PyInstaller)

`app/GUI_Watermark.spec` define cómo PyInstaller construye el ejecutable (el `.spec` se movió a la carpeta `app/` junto con el código fuente). Los puntos clave:

```
a = Analysis(
    ['GUI_Watermark.py'],
    datas=[
        ('rec.jpg', '.'),
    ],
    ...
)
...
exe = EXE(..., exclude_binaries=True, name='GUI_Watermark', console=False, ...)
coll = COLLECT(exe, a.binaries, a.datas, ..., name='GUI_Watermark')
```

`datas` le indica a PyInstaller que copie `rec.jpg` dentro del paquete final. A diferencia de versiones anteriores, ya **no** se incluye la carpeta `poppler-0.68.0` porque el motor de marca de agua actual no la necesita: el ejecutable resultante es más pequeño y no arrastra binarios nativos de terceros. `console=False` evita que se abra una ventana de consola detrás de la interfaz gráfica. El modo de construcción es "onedir" (COLLECT): el resultado es una carpeta (`dist/GUI_Watermark/`) con el `.exe` y sus dependencias.

Comando de reconstrucción (desde la raíz del proyecto): `python -m PyInstaller app/GUI_Watermark.spec --noconfirm`. Al terminar, el contenido de `dist/GUI_Watermark/` (el `.exe` + `_internal/`) se copia para reemplazar el de `GUI_Watermark ejecutable/`, dejando esa carpeta lista para distribuir sin que el usuario final necesite Python.

15. Solución de problemas

Síntoma	Causa probable	Solución
El <code>.exe</code> no abre / cierra inmediatamente.	Se movió o copió solo el <code>.exe</code> , sin la carpeta <code>_internal</code> .	Vuelve a descomprimir/copiar la carpeta "GUI_Watermark ejecutable" completa.
Error al procesar un PDF específico dentro de un lote.	El PDF está dañado, cifrado o es un archivo no válido.	El resto del lote se procesa igualmente; revisa el mensaje de error mostrado al final para identificar el archivo y ábrelo manualmente para confirmarlo.

Síntoma	Causa probable	Solución
La marca de agua se ve muy tenue o muy fuerte.	Valor de opacidad inadecuado, o la imagen ya tiene transparencia propia.	Ajusta el slider de opacidad (0–255) antes de procesar; para más control, edita el canal alfa de la imagen origen.
El texto del PDF de salida ya no se puede seleccionar/copiar.	No debería ocurrir en la versión actual.	Desde que addWaterMarkToPDF dejó de rasterizar la página completa, el texto y los vectores originales se conservan intactos; si ves este problema, confirma que estás usando la versión actual del .exe (sin Poppler).
ModuleNotFoundError: No module named 'PyQt6' (o PyPDF2/PIL/reportlab) al ejecutar desde código fuente.	El intérprete de Python que ejecuta el script no tiene las dependencias instaladas (frecuente si hay más de un Python instalado en la máquina).	Instala las dependencias en ese mismo intérprete: <code>python -m pip install PyQt6 PyPDF2 Pillow reportlab</code> .